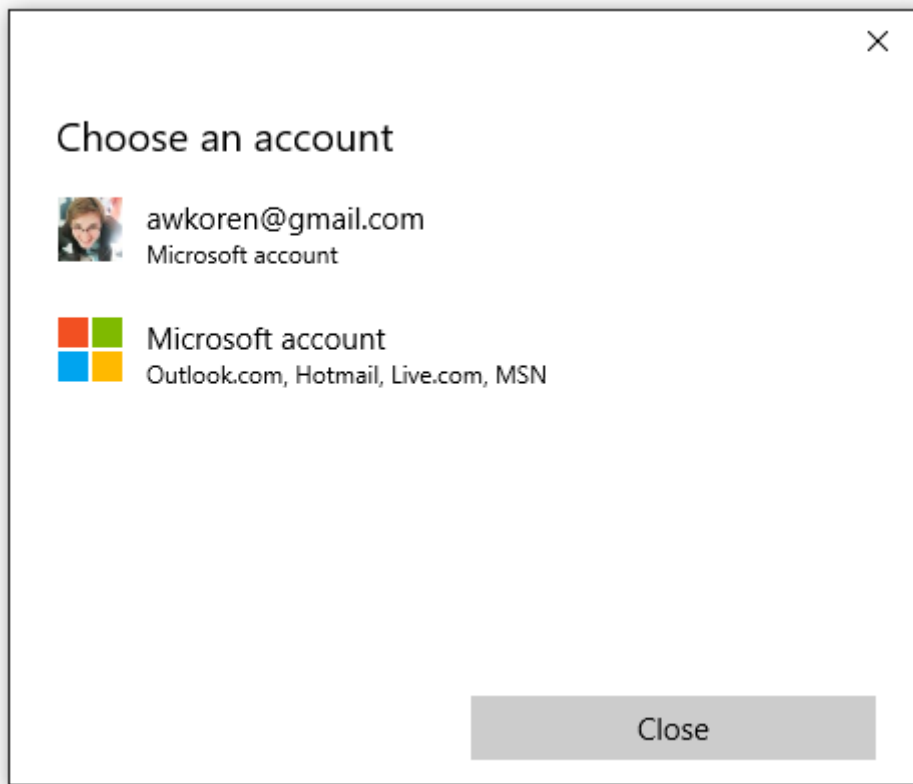




Request a token

Once we have the Microsoft Account option displaying in the **AccountsSettingsPane**, we need to handle what happens when the user selects it. We registered our `GetMsaToken` method to fire when the user chooses to log in with their Microsoft Account, so we'll obtain the token there.

To obtain a token, use the `RequestTokenAsync` method like this:

C#

```
private async void GetMsaTokenAsync(WebAccountProviderCommand command)
{
    WebTokenRequest request = new
WebTokenRequest(command.WebAccountProvider, "wl.basic");
    WebTokenRequestResult result = await
WebAuthenticationCoreManager.RequestTokenAsync(request);
}
```

In this example, we pass the string "wl.basic" to the *scope* parameter. Scope represents the type of information you are requesting from the providing service on a specific user. Certain scopes provide access only to a user's basic information, like name and email address, while other scopes might grant access to sensitive information such as the user's photos or email inbox. Generally, your app should use the least permissive scope

necessary to achieve its function. Service providers will provide documentation on which scopes are needed to get tokens for use with their services.

- For Microsoft 365 and Outlook.com scopes, see [Use the Outlook REST API \(version 2.0\)](#).
- For OneDrive scopes, see [OneDrive authentication and sign-in](#).

Tip

Optionally, if your app uses a login hint (to populate the user field with a default email address) or other special property related to the sign-in experience, list it in the [WebTokenRequest.AppProperties](#) property. This will cause the system to ignore the property when caching the web account, which prevents account mismatches in the cache.

If you're developing an enterprise app, you'll likely want to connect to an Azure Active Directory (AAD) instance and use the Microsoft Graph API instead of regular MSA services. In this scenario, use the following code instead:

C#

```
private async void GetAadTokenAsync(WebAccountProviderCommand command)
{
    string clientId = "your_guid_here"; // Obtain your clientId from the
    Azure Portal
    WebTokenRequest request = new WebTokenRequest(provider, "User.Read",
    clientId);
    request.Properties.Add("resource", "https://graph.microsoft.com");
    WebTokenRequestResult result = await
    WebAuthenticationCoreManager.RequestTokenAsync(request);
}
```

The rest of this article continues describing the MSA scenario, but the code for AAD is very similar. For more info on AAD/Graph, including a full sample on GitHub, see the [Microsoft Graph documentation](#).

Use the token

The RequestTokenAsync method returns a WebTokenRequestResult object, which contains the results of your request. If your request was successful, it will contain a token.

C#

```
private async void GetMsaTokenAsync(WebAccountProviderCommand command)
{
    WebTokenRequest request = new
WebTokenRequest(command.WebAccountProvider, "wl.basic");
    WebTokenRequestResult result = await
WebAuthenticationCoreManager.RequestTokenAsync(request);

    if (result.ResponseStatus == WebTokenRequestStatus.Success)
    {
        string token = result.ResponseData[0].Token;
    }
}
```

ⓘ Note

If you receive an error when requesting a token, make sure you've associated your app with the Store as described in step one. Your app won't be able to get a token if you skipped this step.

Once you have a token, you can use it to call your provider's API. In the code below, we'll call the [user info Microsoft Live API](#) to obtain basic information about the user and display it in our UI. Note however that in most cases it is recommended that you store the token once obtained and then use it in a separate method.

C#

```
private async void GetMsaTokenAsync(WebAccountProviderCommand command)
{
    WebTokenRequest request = new
WebTokenRequest(command.WebAccountProvider, "wl.basic");
    WebTokenRequestResult result = await
WebAuthenticationCoreManager.RequestTokenAsync(request);

    if (result.ResponseStatus == WebTokenRequestStatus.Success)
    {
        string token = result.ResponseData[0].Token;

        var restApi = new Uri(@"https://apis.live.net/v5.0/me?access_token="
+ token);

        using (var client = new HttpClient())
        {
            var infoResult = await client.GetAsync(restApi);
            string content = await infoResult.Content.ReadAsStringAsync();

            var jsonObject = JsonObject.Parse(content);
            string id = jsonObject["id"].GetString();
            string name = jsonObject["name"].GetString();
        }
    }
}
```

```

        UserIdTextBlock.Text = "Id: " + id;
        UserNameTextBlock.Text = "Name: " + name;
    }
}

```

How you call various REST APIs varies between providers; see the provider's API documentation for information on how to use your token.

Store the account for future use

Tokens are useful for immediately obtaining information about a user, but they usually have varying lifespans - MSA tokens, for instance, are only valid for a few hours. Fortunately, you don't need to re-show the **AccountsSettingsPane** each time a token expires. Once a user has authorized your app once, you can store the user's account information for future use.

To do this, use the [WebAccount](#) class. A **WebAccount** is returned by the same method you used to request the token:

C#

```

private async void GetMsaTokenAsync(WebAccountProviderCommand command)
{
    WebTokenRequest request = new
    WebTokenRequest(command.WebAccountProvider, "wl.basic");
    WebTokenRequestResult result = await
    WebAuthenticationCoreManager.RequestTokenAsync(request);

    if (result.ResponseStatus == WebTokenRequestStatus.Success)
    {
        WebAccount account = result.ResponseData[0].WebAccount;
    }
}

```

Once you have a **WebAccount** instance, you can easily store it. In the following example, we use `LocalSettings`. For more information on using `LocalSettings` and other methods to store user data, see [Store and retrieve app settings and data](#).

C#

```

private async void StoreWebAccount(WebAccount account)
{
    ApplicationData.Current.LocalSettings.Values["CurrentUserProviderId"] =
    account.WebAccountProvider.Id;
    ApplicationData.Current.LocalSettings.Values["CurrentUserId"] =

```

```
account.Id;  
}
```

Then, we can use an asynchronous method like the following to attempt to obtain a token in the background with the stored **WebAccount**.

C#

```
private async Task<string> GetTokenSilentlyAsync()  
{  
    string providerId =  
ApplicationData.Current.LocalSettings.Values["CurrentUserProviderId"]?.ToString();  
    string accountId =  
ApplicationData.Current.LocalSettings.Values["CurrentUserId"]?.ToString();  
  
    if (null == providerId || null == accountId)  
    {  
        return null;  
    }  
  
    WebAccountProvider provider = await  
WebAuthenticationCoreManager.FindAccountProviderAsync(providerId);  
    WebAccount account = await  
WebAuthenticationCoreManager.FindAccountAsync(provider, accountId);  
  
    WebTokenRequest request = new WebTokenRequest(provider, "wl.basic");  
  
    WebTokenRequestResult result = await  
WebAuthenticationCoreManager.GetTokenSilentlyAsync(request, account);  
    if (result.ResponseStatus ==  
WebTokenRequestStatus.UserInteractionRequired)  
    {  
        // Unable to get a token silently - you'll need to show the UI  
        return null;  
    }  
    else if (result.ResponseStatus == WebTokenRequestStatus.Success)  
    {  
        // Success  
        return result.ResponseData[0].Token;  
    }  
    else  
    {  
        // Other error  
        return null;  
    }  
}
```

Place the above method just before the code that builds the **AccountsSettingsPane**. If the token is obtained in the background, there is no need to show the pane.

C#

```
private void LoginButton_Click(object sender, RoutedEventArgs e)
{
    string silentToken = await GetMsaTokenSilentlyAsync();

    if (silentToken != null)
    {
        // the token was obtained. store a reference to it or do something
        with it here.
    }
    else
    {
        // the token could not be obtained silently. Show the
        AccountsSettingsPane
        AccountsSettingsPane.Show();
    }
}
```

Because obtaining a token silently is very simple, you should use this process to refresh your token between sessions rather than caching an existing token (since that token might expire at any time).

ⓘ Note

The example above only covers basic success and fail cases. Your app should also account for unusual scenarios (like a user revoking your app's permission or removing their account from Windows, for example) and handle them gracefully.

Remove a stored account

If you persist a web account, you may want to give your users the ability to disassociate their account with your app. This way, they can effectively "log out" of the app: their account information will no longer be loaded automatically upon launch. To do this, first remove any saved account and provider information from storage. Then call [SignOutAsync](#) to clear the cache and invalidate any existing tokens your app may have.

C#

```
private async Task SignOutAccountAsync(WebAccount account)
{
    ApplicationData.Current.LocalSettings.Values.Remove("CurrentUserProviderId");
    ;
    ApplicationData.Current.LocalSettings.Values.Remove("CurrentUserId");
}
```

```
account.SignOutAsync();  
}
```

Add providers that don't support WebAccountManager

If you want to integrate authentication from a service into your app but that service doesn't support WebAccountManager - Google+ or Twitter, for example - you can still manually add that provider to the **AccountsSettingsPane**. To do so, create a new WebAccountProvider object and provide your own name and .png icon, then add it to the WebAccountProviderCommands list. Here's some stub code:

C#

```
private async void BuildPaneAsync(AccountsSettingsPane s,  
AccountsSettingsPaneCommandsRequestedEventArgs e)  
{  
    // other code here  
  
    var twitterProvider = new WebAccountProvider("twitter", "Twitter", new  
Uri(@"ms-appx:///Assets/twitter-auth-icon.png"));  
    var twitterCmd = new WebAccountProviderCommand(twitterProvider,  
GetTwitterTokenAsync);  
    e.WebAccountProviderCommands.Add(twitterCmd);  
  
    // other code here  
}  
  
private async void GetTwitterTokenAsync(WebAccountProviderCommand command)  
{  
    // Manually handle Twitter login here  
}
```

⚠ Note

This only adds an icon to the **AccountsSettingsPane** and runs the method you specify when the icon is clicked (GetTwitterTokenAsync, in this case). You must provide the code that handles the actual authentication. For more information, see [Web authentication broker](#), which provides helper methods for authenticating using REST services.

Add a custom header

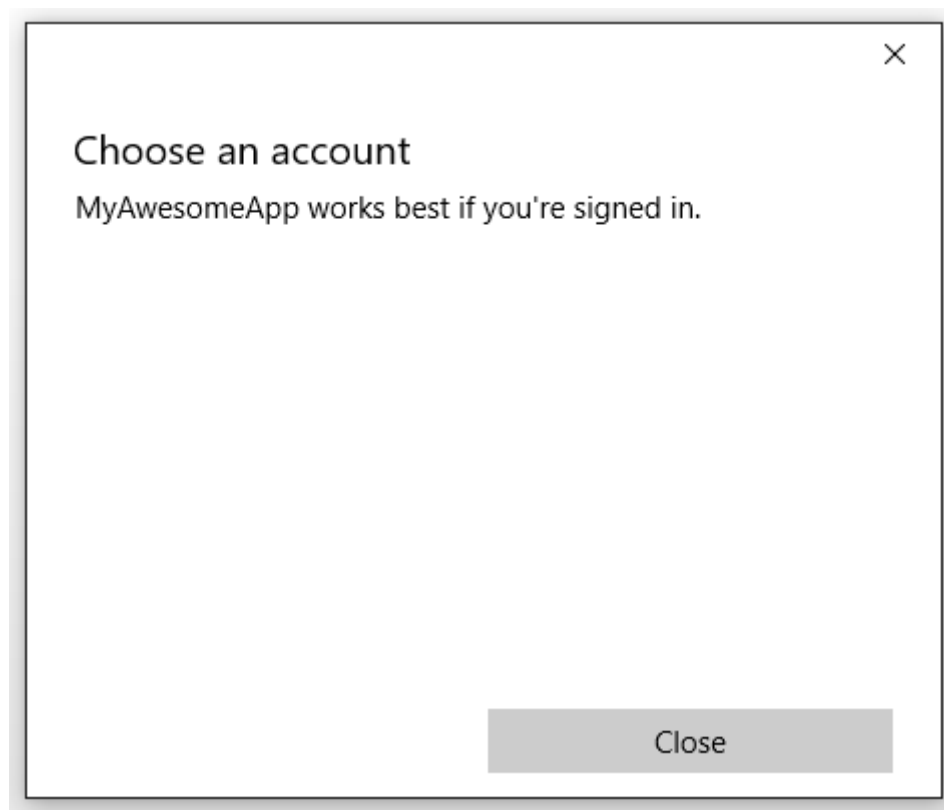
You can customize the account settings pane using the `HeaderText` property, like this:

C#

```
private async void BuildPaneAsync(AccountsSettingsPane s,
AccountsSettingsPaneCommandsRequestedEventArgs e)
{
    // other code here

    args.HeaderText = "MyAwesomeApp works best if you're signed in.";

    // other code here
}
```



Don't go overboard with header text; keep it short and sweet. If your login process is complicated and you need to display more information, link the user to a separate page using a custom link.

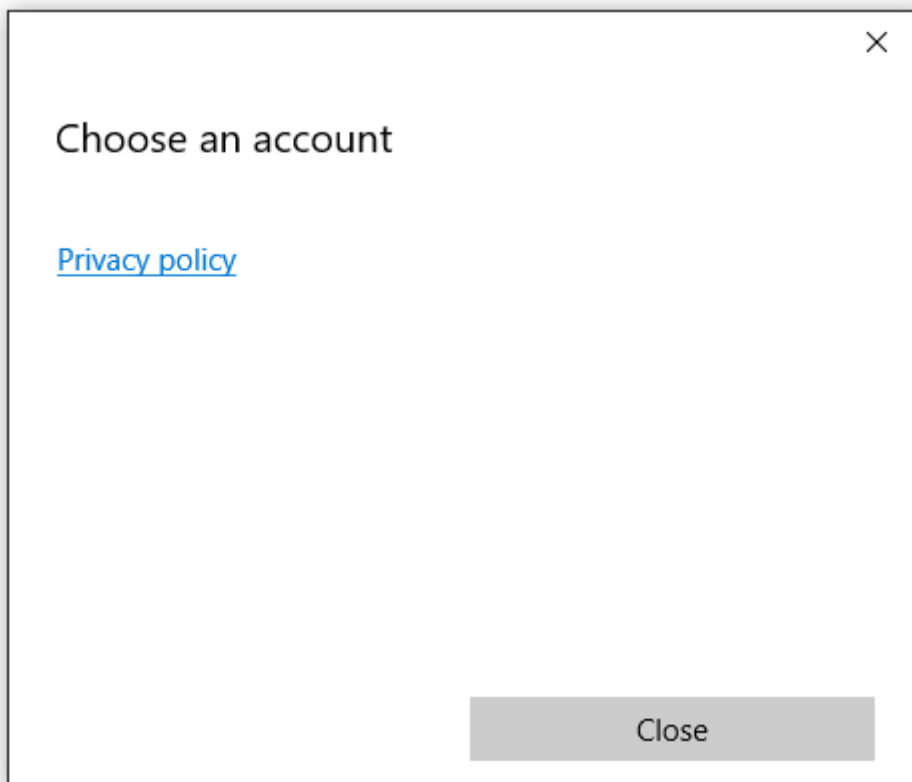
Add custom links

You can add custom commands to the `AccountsSettingsPane`, which appear as links below your supported `WebAccountProviders`. Custom commands are great for simple tasks related to user accounts, like displaying a privacy policy or launching a support page for users having trouble.

Here's an example:

C#

```
private async void BuildPaneAsync(AccountsSettingsPane s,  
AccountsSettingsPaneCommandsRequestedEventArgs e)  
{  
    // other code here  
  
    var settingsCmd = new SettingsCommand(  
        "settings_privacy",  
        "Privacy policy",  
        async (x) => await Launcher.LaunchUriAsync(new  
Uri(@"https://privacy.microsoft.com/en-US/")));  
  
    e.Commands.Add(settingsCmd);  
  
    // other code here  
}
```



Theoretically, you can use settings commands for anything. However, we suggest limiting their use to intuitive, account-related scenarios like those described above.

See also

[Windows.Security.Authentication.Web.Core namespace](#)

[Windows.Security.Credentials namespace](#)

[AccountsSettingsPane class](#)

[Web authentication broker](#)

[Web account management sample](#) 

[Lunch Scheduler app](#) 

Feedback

Was this page helpful?

 **Yes**

 **No**

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Windows Unlock with Windows Hello companion (IoT) devices

Article • 10/20/2022

A Windows Hello companion device is a device that can act in conjunction with your Windows 10 desktop to enhance the user authentication experience. Using the Windows Hello companion device framework, a companion device can provide a rich experience for Windows Hello even when biometrics are not available (for example, if the Windows 10 desktop lacks a camera for face authentication or fingerprint reader device, for example).

ⓘ Note

The API for the Windows Hello companion device framework is deprecated in Windows 10, version 2004.

Introduction

For code samples, see the [Windows Hello companion device framework Github repository](#) .

Use cases

There are numerous ways one can use the Windows Hello companion device framework to build a great Windows unlock experience with a companion device. For example, users could:

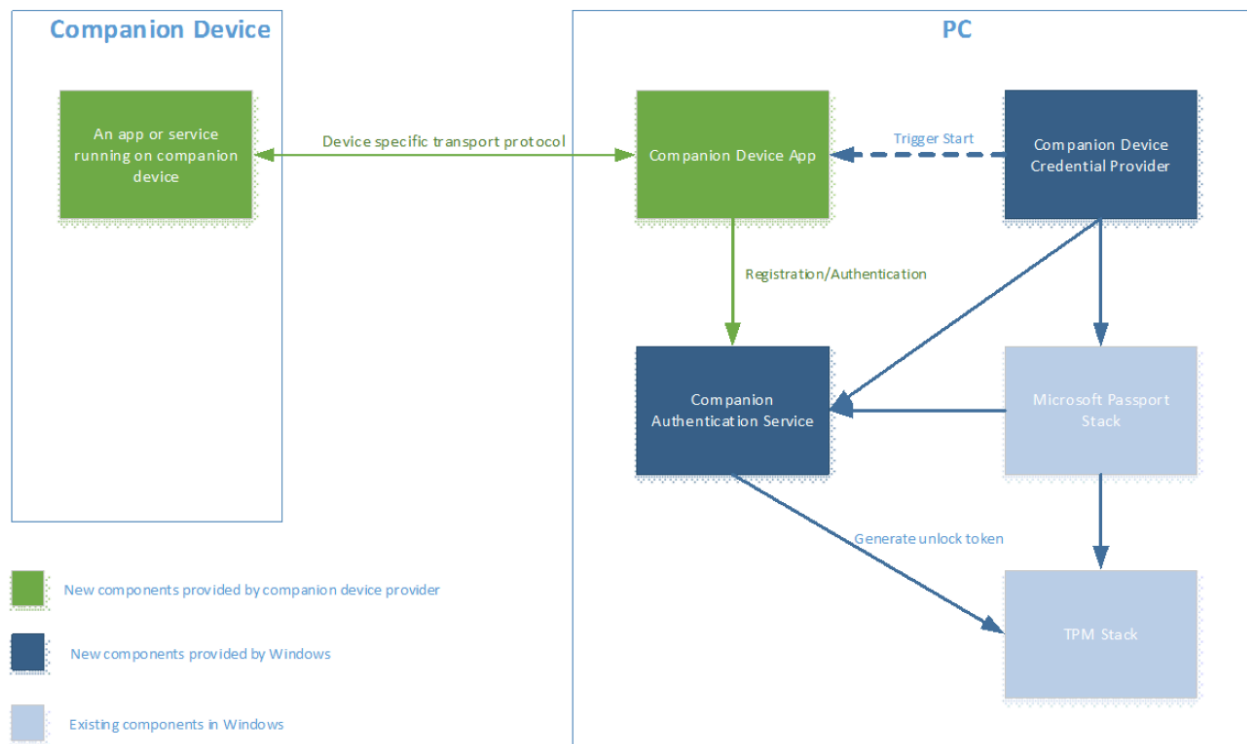
- Attach their companion device to PC via USB, touch the button on the companion device, and automatically unlock their PC.
- Carry a phone in their pocket that is already paired with PC over Bluetooth. Upon hitting the spacebar on their PC, their phone receives a notification. Approve it and the PC simply unlocks.
- Tap their companion device to an NFC reader to quickly unlock their PC.
- Wear a fitness band that has already authenticated the wearer. Upon approaching PC, and by performing a special gesture (like clapping), the PC unlocks.

Biometric enabled Windows Hello companion devices

If the companion device supports biometrics, in some cases the [Windows Biometric framework](#) may be a better solution than the Windows Hello companion device framework.

Components of the solution

The diagram below depicts the components of the solution and who is responsible for building them.



The Windows Hello companion device framework is implemented as a service running on Windows (called the Companion Authentication Service in this article). This service is responsible for generating an unlock token which needs to be protected by an HMAC key stored on the Windows Hello companion device. This guarantees that access to the unlock token requires Windows Hello companion device presence. Per each (PC, Windows user) tuple, there will be a unique unlock token.

Integration with the Windows Hello Companion Device Framework requires:

- A [Universal Windows Platform \(UWP\)](#) Windows Hello companion device app for the companion device, downloaded from the Windows app store.
- The ability to create two 256 bit HMAC keys on the Windows Hello companion device and generate HMAC with it (using SHA-256).
- Security settings on the Windows 10 desktop properly configured. The Companion Authentication Service will require this PIN to be set up before any Windows Hello companion device can be plugged into it. The users must set up a PIN via Settings > Accounts > Sign-in options.

In addition to the above requirements, the Windows Hello companion device app is responsible for:

- User experience and branding of initial registration and later de-registration of the Windows Hello companion device.
- Running in the background, discovering the Windows Hello companion device, communicating to the Windows Hello companion device and also Companion Authentication Service.
- Error handling

Normally, companion devices ship with an app for initial setup, like setting up a fitness band for the first time. The functionality described in this document can be part of that app and a separate app should not be required.

User signals

Each Windows Hello companion device should be combined with an app that supports three user signals. These signals can be in form of an action or gesture.

- **Intent signal:** Allows the user to show his intent for unlock by, for example, hitting a button on the Windows Hello companion device. The intent signal must be collected on **Windows Hello companion device** side.
- **User presence signal:** Proves the presence of the user. The Windows Hello companion device might, for instance, require a PIN before it can be used for unlocking PC (not to be confused with PC PIN), or it might require press of a button.
- **Disambiguation signal:** Disambiguates which Windows 10 desktop the user wants to unlock when multiple options are available to the Windows Hello companion device.

Any number of these user signals can be combined into one. User presence and intent signals must be required on each use.

Registration and future communication between a PC and Windows Hello companion devices

Before a Windows Hello companion device can be plugged into the Windows Hello companion device framework, it needs to be registered with the framework. The experience for registration is completely owned by the Windows Hello companion device app.

The relationship between the Windows Hello companion device and the Windows 10 desktop device can be one to many (i.e., one companion device can be used for many Windows 10 desktop devices). However, each Windows Hello companion device can only be used for one user on each Windows 10 desktop device.

Before a Windows Hello companion device can communicate with a PC, they need to agree on a transport to use. Such choice is left to the Windows Hello companion device app; the Windows Hello companion device framework does not impose any limitations on transport type (USB, NFC, WiFi, BT, BLE, etc) or protocol being used between the Windows Hello companion device and the Windows Hello companion device app on the Windows 10 desktop device side. It does, however, suggest certain security considerations for the transport layer as outlined in the "Security Requirements" section of this document. It is the device provider's responsibility to provide those requirements. The framework does not provide them for you.

User Interaction Model

Windows Hello companion device app discovery, installation, and first-time registration

A typical user workflow is as follows:

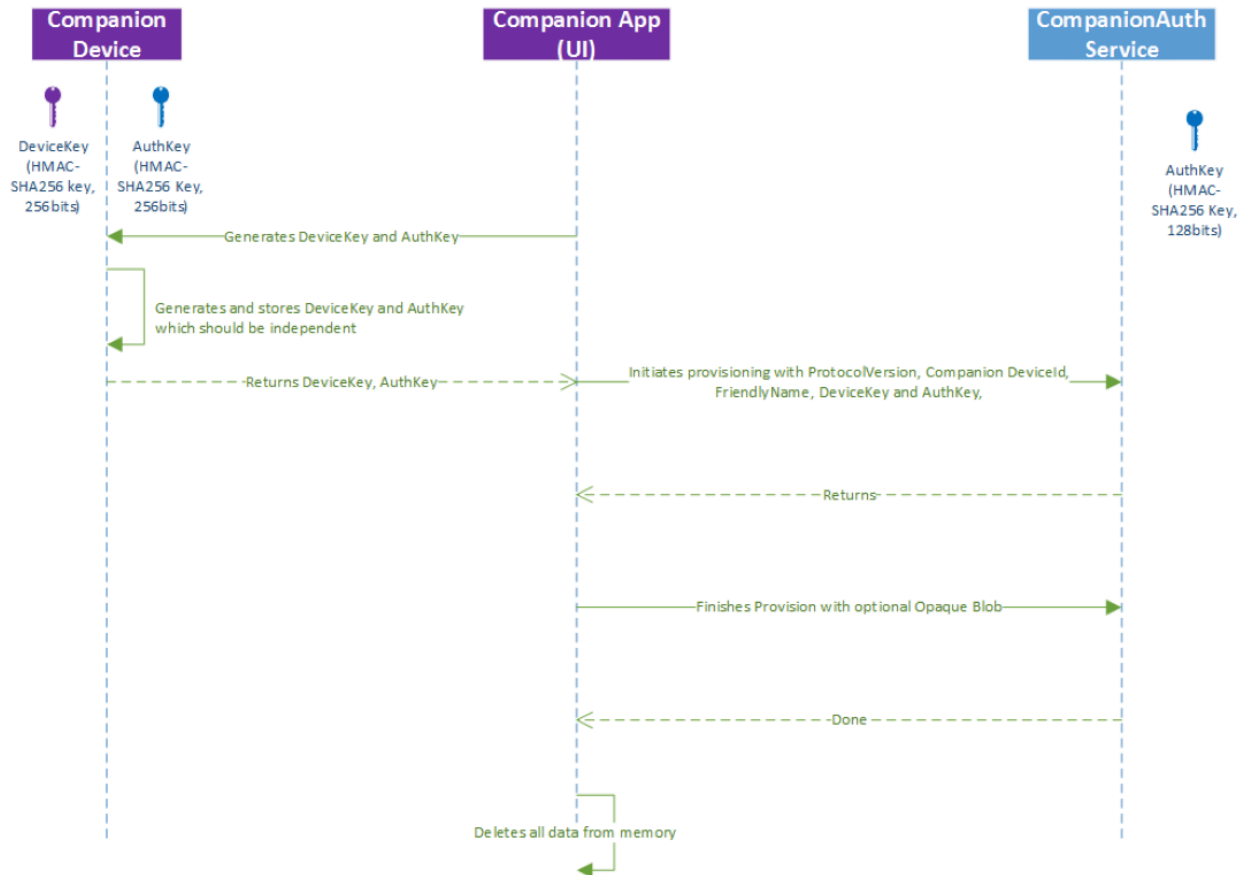
- The user sets up the PIN on each of target Windows 10 desktop devices she wants to unlock with that Windows Hello companion device.
- The user runs the Windows Hello companion device app on their Windows 10 desktop device to register her Windows Hello companion device with Windows 10 desktop.

Notes:

- We recommend the discovery, download, and launch of the Windows Hello companion device app is streamlined and, if possible, automated (for example, the app can be downloaded upon tapping the Windows Hello companion device on an NFC reader on Windows 10 desktop device side). This is, however, the responsibility of the Windows Hello companion device and Windows Hello companion device app.
- In an enterprise environment, the Windows Hello companion device app can be deployed via MDM.
- The Windows Hello companion device app is responsible for showing the user any error messages that happen as part of registration.

Registration and de-registration protocol

The following diagram illustrates how the Windows Hello companion device interacts with Companion Authentication Service during registration.



There are two keys used in our protocol:

- Device key (**devicekey**): used to protect unlock tokens that the PC needs to unlock Windows.
- The authentication key (**authkey**): used to mutually authenticate the Windows Hello companion device and Companion Authentication Service.

The device key and authentication keys are exchanged at registration time between the Windows Hello companion device app and Windows Hello companion device. As a result, the Windows Hello companion device app and Windows Hello companion device must use a secure transport to protect keys.

Also, note that while the diagram above displays two HMAC keys generating on the Windows Hello companion device, it is also possible for the app to generate them and send them to the Windows Hello companion device for storage.

Starting authentication flows

There are two ways for the user to start the signing in flow to Windows 10 desktop using Windows Hello companion device framework (i.e., provide intent signal):

- Open up the lid on laptop, or hit the space bar or swipe up on PC.
- Perform a gesture or an action on the Windows Hello companion device side.

It is the Windows Hello companion device's choice to select which one is the starting point. The Windows Hello companion device framework will inform companion device app when option one happens. For option two, the Windows Hello companion device app should query the companion device to see if that event has been captured. This ensures the Windows Hello companion device collects the intent signal before the unlock succeeds.

Windows Hello companion device credential provider

There is a new credential provider in Windows 10 that handles all Windows Hello companion devices.

The Windows Hello companion device credential provider is responsible for launching the companion device background task via activating a trigger. The trigger is set the first time when the PC awakens and a lock screen is displayed. The second time is when the PC is entering logon UI and the Windows Hello companion device credential provider is the selected tile.

The helper library for the Windows Hello companion device app will listen to the lock screen status change and send the event corresponding to the Windows Hello companion device background task.

If there are multiple Windows Hello companion device background tasks, the first background task that has finished the authentication process will unlock the PC. The companion device authentication service will ignore any remaining authentication calls.

The experience on the Windows Hello companion device side is owned and managed by the Windows Hello companion device app. The Windows Hello companion device framework has no control over this part of the user experience. More specifically, the companion authentication provider informs the Windows Hello companion device app (via its background app) about state changes in logon UI (for example, lock screen just came down, or the user just dispelled lock screen by hitting spacebar), and it is the responsibility of the Windows Hello companion device app to build an experience around that (for example, upon user hitting spacebar and dispelling unlock screen, start looking for the device over USB).

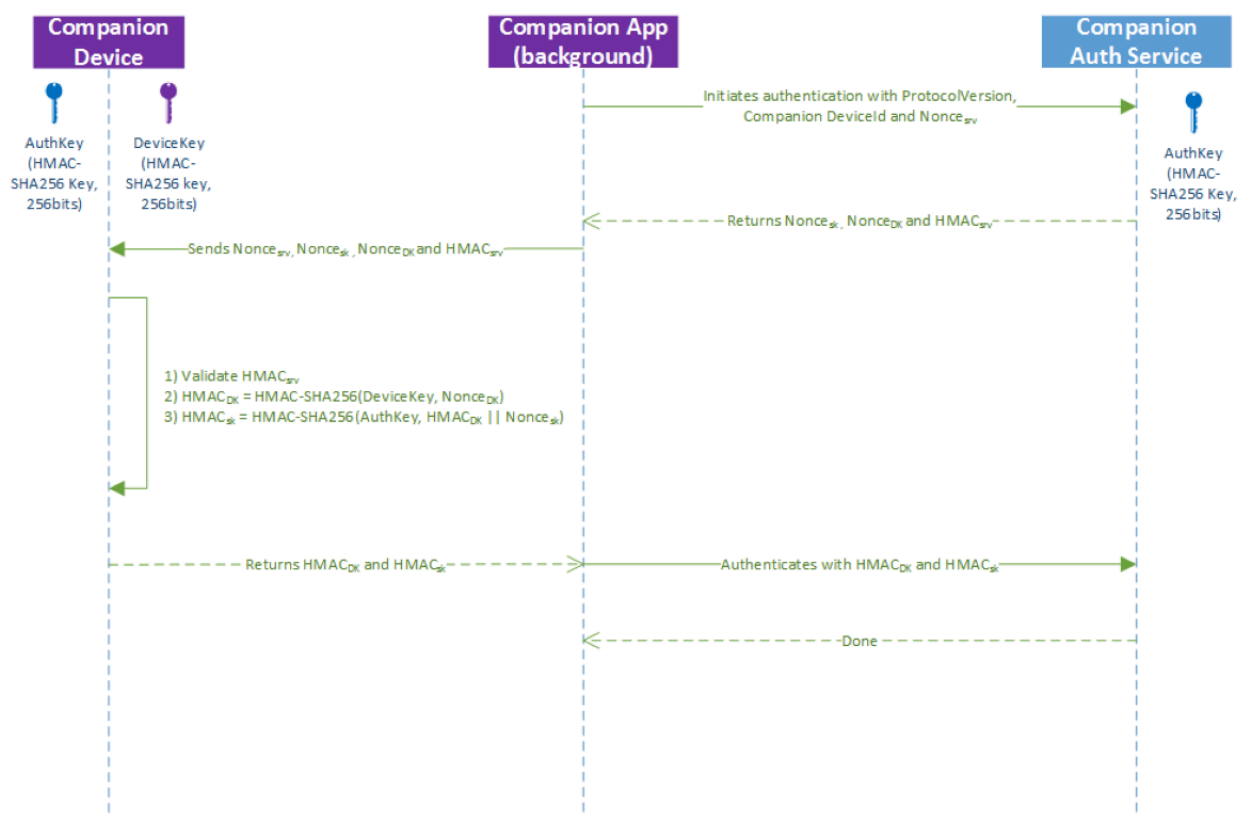
The Windows Hello companion device Framework will provide a stock of (localized) text and error messages for the Windows Hello companion device app to choose from. These will be displayed on top of lock screen (or in login UI). See the Dealing with Messages and Errors section for more details.

Authentication protocol

Once the background task associated with a Windows Hello companion device app is trigger started, it is responsible for asking the Windows Hello companion device to validate an HMAC value computed by the Companion Authentication Service and help calculate two HMAC values:

- Validate Service HMAC = HMAC(authentication key, service nonce || device nonce || session nonce).
- Calculate the HMAC of the device key with a nonce.
- Calculate the HMAC of the authentication key with first HMAC value concatenated with a nonce generated by the Companion Authentication Service.

The second computed value is used by the service to authenticate the device and also prevent replay attack in transport channel.



Lifecycle management

Register once, use everywhere

Without a backend server, users must register their Windows Hello companion device with each Windows 10 desktop device separately.

A companion device vendor or OEM can implement a web service to roam the registration state across user Windows 10 desktops or mobile devices. For more details, see the Roaming, Revocation, and Filter Service section.

PIN management

Before a companion device can be used, a PIN needs to be set up on Windows 10 desktop device. This ensures the user has a backup in case their Windows Hello companion device is not working. The PIN is something that Windows manages and that apps never see. To change it, the user navigates to Settings > Accounts > Sign-in options.

Management and policy

Users can remove a Windows Hello companion device from a Windows 10 desktops by running the Windows Hello companion device app on that desktop device.

Enterprises have two options for controlling the Windows Hello companion device framework:

- Turn the feature on or off
- Define the allowlist of Windows Hello companion devices allowed using Windows app locker

The Windows Hello companion device framework does not support any centralized way to keep inventory of available companion devices, or a method to further filter which instances of a Windows Hello companion device type is allowed (for example, only a companion device with a serial number between X and Y are allowed). Apps developers can, however, build a service to provide such functionality. For more details, see the Roaming, Revocation, and Filter Service section.

Revocation

The Windows Hello companion device framework does not support removing a companion device from a specific Windows 10 desktop device remotely. Instead, users can remove the Windows Hello companion device via the Windows Hello companion device app running on that Windows 10 desktop.

Companion device vendors, however, can build a service to provide remote revocation functionality. For more details, see Roaming, Revocation, and Filter Service section.

Roaming and filter services

Companion device vendors can implement a web service that can be used for the following scenarios:

- A filter service for enterprise: An enterprise can limit the set of Windows Hello companion devices that can work in their environment to a select few from a specific vendor. For example, the company Contoso could order 10,000 Model Y companion devices from Vendor X and ensure only those devices will work in the Contoso domain (and not any other device model from Vendor X).
- Inventory: An enterprise can determine the list of existing companion devices used in an enterprise environment.
- Real time revocation: If an employee reports that his companion device is lost or stolen, the web service can be used to revoke that device.
- Roaming: A user only has to register his companion device once and it works on all of his Windows 10 desktops and Mobile.

Implementing these features requires the Windows Hello companion device app to check with the web service at registration and usage time. The Windows Hello companion device app can optimize for cached logon scenarios like requiring checking with web service only once a day (at the cost of extending the revocation time to up to one day).

Windows Hello companion device framework API model

Overview

A Windows Hello companion device app should contain two components: a foreground app with UI responsible for registering and unregistering the device, and a background task that handles authentication.

The overall API flow is as follows:

1. Register the Windows Hello companion device

- Make sure the device is nearby and query its capability (if required)

- Generate two HMAC keys (either on the companion device side or the app side)
 - Call `RequestStartRegisteringDeviceAsync`
 - Call `FinishRegisteringDeviceAsync`
 - Make sure Windows Hello companion device app stores HMAC keys (if supported) and Windows Hello companion device app discards its copies
2. Register your background task
 3. Wait for the right event in the background task
 - `WaitingForUserConfirmation`: Wait for this event if the user action/gesture on the Windows Hello companion device side is required to start authentication flow
 - `CollectingCredential`: Wait for this event if the Windows Hello companion device relies on user action/gesture on the PC side to start authentication flow (for example, by hitting spacebar)
 - Other trigger, like a smartcard: Make sure to query for current authentication state to call the right APIs.
 4. Keep user informed about error messages or required next steps by calling `ShowNotificationMessageAsync`. Only call this API once an intent signal is collected
 5. Unlock
 - Make sure intent and user presence signals were collected
 - Call `StartAuthenticationAsync`
 - Communicate with the companion device to perform required HMAC operations
 - Call `FinishAuthenticationAsync`
 6. Un-register a Windows Hello companion device when the user requests it (for example, if they've lost their companion device)
 - Enumerate the Windows Hello companion device for logged in user via `FindAllRegisteredDeviceInfoAsync`
 - Un-register it using `UnregisterDeviceAsync`

Registration and de-registration

Registration requires two API calls to the Companion Authentication Service: `RequestStartRegisteringDeviceAsync` and `FinishRegisteringDeviceAsync`.

Before any of these calls are made, the Windows Hello companion device app must make sure that the Windows Hello companion device is available. If the Windows Hello companion device is responsible for generating HMAC keys (authentication and device

keys), then the Windows Hello companion device app should also ask the companion device to generate them before making any of the above two calls. If the Windows Hello companion device app is responsible for generating HMAC keys, then it should do so before calling the above two calls.

Additionally, as part of first API call (`RequestStartRegisteringDeviceAsync`), the Windows Hello companion device app must decide on device capability and be prepared to pass it as part of the API call; for example, whether the Windows Hello companion device supports secure storage for HMAC keys. If the same Windows Hello companion device app is used to manage multiple versions of the same companion device and those capabilities change (and requires a device query to decide), we recommend this queries occurs before first API call is made.

The first API (`RequestStartRegisteringDeviceAsync`) will return a handle used by the second API (`FinishRegisteringDeviceAsync`). The first call for registration will launch the PIN prompt to make sure user is present. If no PIN is set up, this call will fail. The Windows Hello companion device app can query whether PIN is set up or not via `KeyCredentialManager.IsSupportedAsync` call as well.

`RequestStartRegisteringDeviceAsync` call can also fail if policy has disabled the usage of the Windows Hello companion device.

The result of first call is returned via `SecondaryAuthenticationFactorRegistrationStatus` enum:

```
C#

{
    Failed = 0,           // Something went wrong in the underlying components
    Started,             // First call succeeded
    CanceledByUser,      // User cancelled PIN prompt
    PinSetupRequired,    // PIN is not set up
    DisabledByPolicy,     // Companion device framework or this app is
disabled
}
```

The second call (`FinishRegisteringDeviceAsync`) finishes the registration. As part of registration process, the Windows Hello companion device app can store companion device configuration data with Companion Authentication Service. There is a 4K size limit for this data. This data will be available to the Windows Hello companion device app at authentication time. This data can be used, as an example, to connect to the Windows Hello companion device like a MAC address, or if the Windows Hello companion device does not have storage and companion device wants to use PC for storage, then configuration data can be used. Note that any sensitive data stored as part of configuration data must be encrypted with a key that only the Windows Hello

companion device knows. Also, given that configuration data is stored by a Windows service, it is available to the Windows Hello companion device app across user profiles.

The Windows Hello companion device app can call `AbortRegisteringDeviceAsync` to cancel the registration and pass in an error code. The Companion Authentication Service will log the error in the telemetry data. A good example for this call would be when something went wrong with the Windows Hello companion device and it could not finish registration (for example, it cannot store HMAC keys or BT connection was lost).

The Windows Hello companion device app must provide an option for the user to de-register their Windows Hello companion device from their Windows 10 desktop (for example, if they lost their companion device or bought a newer version). When the user selects that option, then the Windows Hello companion device app must call `UnregisterDeviceAsync`. This call by the Windows Hello companion device app will trigger the companion device authentication service to delete all data (including HMAC keys) corresponding to the specific device Id and AppId of the caller app from PC side. This API call does not attempt to delete HMAC keys from either the Windows Hello companion device app or companion device side. That is left for the Windows Hello companion device app to implement.

The Windows Hello companion device app is responsible for showing any error messages that happen in registration and de-registration phase.

C#

```
using System;
using Windows.Security.Authentication.Identity.Provider;
using Windows.Storage.Streams;
using Windows.Security.Cryptography;
using Windows.UI.Popups;

namespace SecondaryAuthFactorSample
{
    public class DeviceRegistration
    {
        public void async OnRegisterButtonClick()
        {
            //
            // Pseudo function, the deviceId should be retrieved by the
            application from the device
            //
            string deviceId = await ReadSerialNumberFromDevice();

            IBuffer deviceKey = CryptographicBuffer.GenerateRandom(256/8);
            IBuffer mutualAuthenticationKey =
            CryptographicBuffer.GenerateRandom(256/8);
```

```

        SecondaryAuthenticationFactorRegistration registrationResult =
            await
SecondaryAuthenticationFactorRegistration.RequestStartRegisteringDeviceAsync
(
    deviceId, // deviceId: max 40 wide characters. For
example, serial number of the device

SecondaryAuthenticationFactorDeviceCapabilities.SecureStorage |

SecondaryAuthenticationFactorDeviceCapabilities.HMacSha256 |

SecondaryAuthenticationFactorDeviceCapabilities.StoreKeys,
    "My test device 1", // deviceFriendlyName: max 64 wide
characters. For example: John's card
    "SAMPLE-001", // deviceModelNumber: max 32 wide
characters. The app should read the model number from device.
    deviceKey,
    mutualAuthenticationKey);

switch(registerResult.Status)
{
case SecondaryAuthenticationFactorRegistrationStatus.Started:
    //
    // Pseudo function:
    // The app needs to retrieve the value from device and set
into opaqueBlob
    //
    IBuffer deviceConfigData =
ReadConfigurationDataFromDevice();

    if (deviceConfigData != null)
    {
        await
registrationResult.Registration.FinishRegisteringDeviceAsync(deviceConfigDat
a); //config data limited to 4096 bytes
        MessageDialog dialog = new MessageDialog("The device is
registered correctly.");
        await dialog.ShowAsync();
    }
    else
    {
        await
registrationResult.Registration.AbortRegisteringDeviceAsync("Failed to
connect to the device");
        MessageDialog dialog = new MessageDialog("Failed to
connect to the device.");
        await dialog.ShowAsync();
    }
    break;

case
SecondaryAuthenticationFactorRegistrationStatus.CanceledByUser:
    MessageDialog dialog = new MessageDialog("You didn't enter
your PIN.");
    await dialog.ShowAsync();

```

```

        break;

        case
SecondaryAuthenticationFactorRegistrationStatus.PinSetupRequired:
            MessageDialog dialog = new MessageDialog("Please setup PIN
in settings.");
            await dialog.ShowAsync();
            break;

        case
SecondaryAuthenticationFactorRegistrationStatus.DisabledByPolicy:
            MessageDialog dialog = new MessageDialog("Your enterprise
prevents using this device to sign in.");
            await dialog.ShowAsync();
            break;
    }
}

public void async UpdateDeviceList()
{
    IReadOnlyList<SecondaryAuthenticationFactorInfo> deviceInfoList
=
        await
SecondaryAuthenticationFactorRegistration.FindAllRegisteredDeviceInfoAsync(
    SecondaryAuthenticationFactorDeviceFindScope.User);

    if (deviceInfoList.Count > 0)
    {
        foreach (SecondaryAuthenticationFactorInfo deviceInfo in
deviceInfoList)
        {
            //
            // Add deviceInfo.FriendlyName and deviceInfo.DeviceId
into a combo box
            //
        }
    }
}

public void async OnUnregisterButtonClick()
{
    string deviceId;
    //
    // Read the deviceId from the selected item in the combo box
    //
    await
SecondaryAuthenticationFactorRegistration.UnregisterDeviceAsync(deviceId);
}
}
}

```

Authentication

Authentication requires two API calls to the Companion Authentication Service: `StartAuthenticationAsync` and `FinishAuthencationAsync`.

The first initiation API will return a handle used by the second API. The first call returns, among other things, a nonce that – once concatenated with other things - needs to be HMAC'ed with the device key stored on the Windows Hello companion device. The second call returns the results of HMAC with device key and can potentially end in successful authentication (i.e., the user will see their desktop).

The first initiation API (`StartAuthenticationAsync`) can fail if policy has disabled that Windows Hello companion device after initial registration. It can also fail if the API call was made outside `WaitingForUserConfirmation` or `CollectingCredential` states (more on this later in this section). It can also fail if an unregistered companion device app calls it. `SecondaryAuthenticationFactorAuthenticationStatus` Enum summarizes the possible outcomes:

```
C#

{
    Failed = 0,                // Something went wrong in the
    underlying components
    Started,
    UnknownDevice,            // Companion device app is not
    registered with framework
    DisabledByPolicy,          // Policy disabled this device after
    registration
    InvalidAuthenticationStage, // Companion device framework is not
    currently accepting
                                // incoming authentication requests
}
```

The second API call (`FinishAuthencationAsync`) can fail if the nonce that was provided in the first call is expired (20 seconds).

`SecondaryAuthenticationFactorFinishAuthenticationStatus` enum captures possible outcomes.

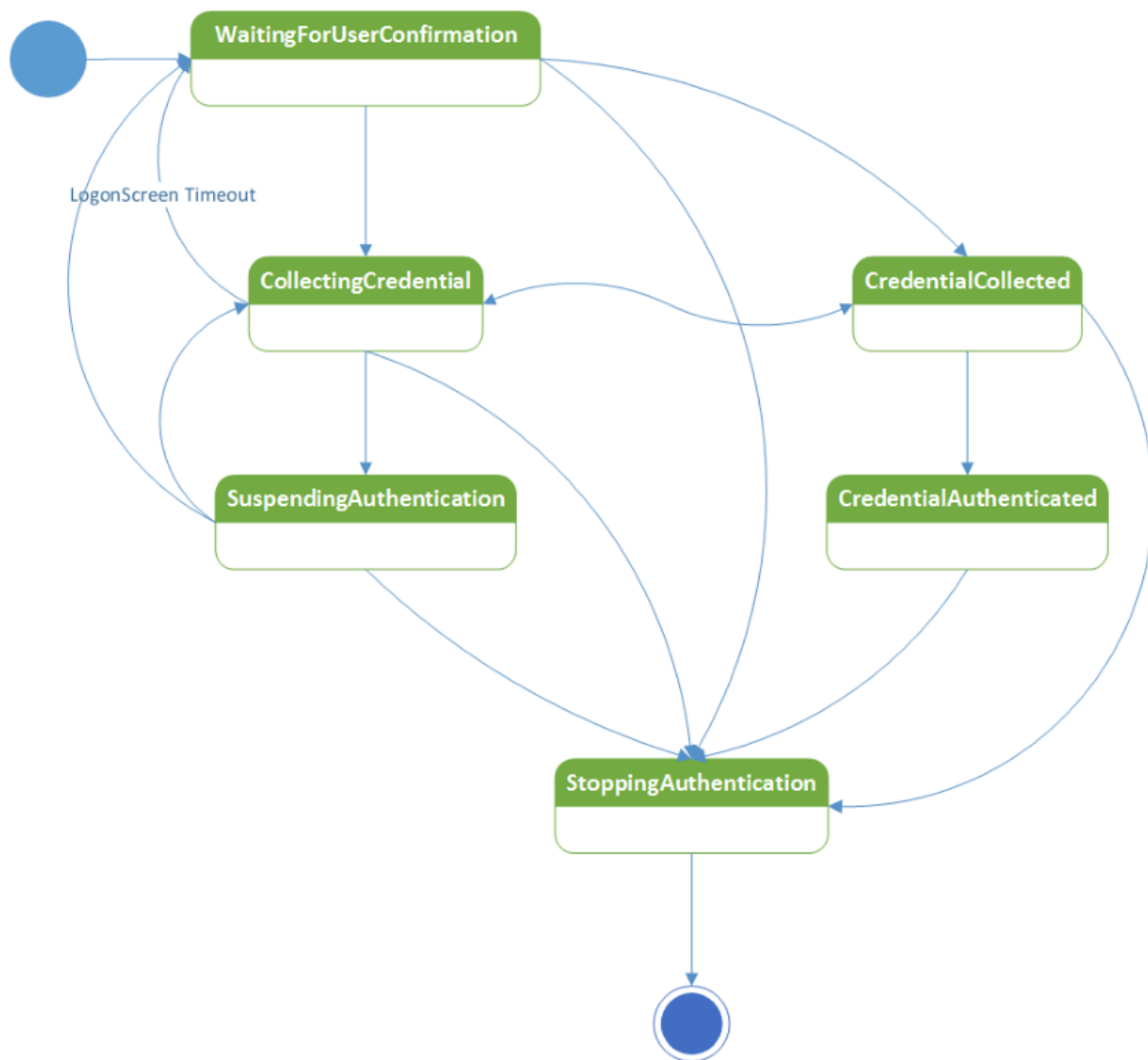
```
C#

{
    Failed = 0,                // Something went wrong in the underlying components
    Completed,                 // Success
    NonceExpired,               // Nonce is expired
}
```

The timing of two API calls (`StartAuthenticationAsync` and `FinishAuthencationAsync`) needs to align with how the Windows Hello companion device collects intent, user

presence, and disambiguation signals (see User Signals for more details). For example, the second call must not be submitted until intent signal is available. In other words, the PC should not unlock if the user has not expressed intent for it. To make this more clear, assume that Bluetooth proximity is used for PC unlock, then an explicit intent signal must be collected, otherwise, as soon as user walks by his PC on the way to kitchen, the PC will unlock. Also, the nonce returned from the first call is time bound (20 seconds) and will expire after certain period. As a result, the first call only should be made when the Windows Hello companion device app has good indication of companion device presence, for example, the companion device is inserted into USB port, or tapped on NFC reader. With Bluetooth, care must be taken to avoid affecting battery on PC side or affecting other Bluetooth activities going on at that point when checking for Windows Hello companion device presence. Also, if a user presence signal needs to be provided (for example, by typing in PIN), it is recommended that the first authentication call is only made after that signal is collected.

The Windows Hello companion device framework helps the Windows Hello companion device app to make informed decision on when to make above two calls by providing a complete picture of where user is in authentication flow. Windows Hello companion device framework provides this functionality by providing lock state change notification to app background task.



Details of each of these states are as follows:

[Expand table](#)

State	Description
WaitingForUserConfirmation	This state change notification event is fired when the lock screen comes down (for example, user pressed Windows + L). We recommend not to request any error messages relating to having difficulty finding a device in this state. In general, we recommend to only show messages when intent signal is available. The Windows Hello companion device app should make the first API call for authentication in this state if the companion device collects the intent signal (for example, tapping on NFC reader, press of a button on the companion device or a specific gesture, like clapping), and the Windows Hello companion device app background task receives indication from the companion device that intent signal was detected. Otherwise, if the Windows Hello companion device app relies on the PC to start authentication flow (by having user swipe up the unlock screen or hitting space bar),

State	Description
	then the Windows Hello companion device app needs to wait for the next state (CollectingCredential).
CollectingCredential	This state change notification event is fired when the user either opens their laptop lid, hits any key on their keyboard, or swipes up to the unlock screen. If the Windows Hello companion device relies on the above actions to start collecting the intent signal, then the Windows Hello companion device app should start collecting it (for example, via a pop up on the companion device asking whether user wants to unlock the PC). This would be a good time to provide error cases if the Windows Hello companion device app needs the user to provide a user presence signal on the companion device (like typing in PIN on the Windows Hello companion device).
SuspendingAuthentication	When the Windows Hello companion device app receives this state, it means that the Companion Authentication Service has stopped accepting authentication requests.
CredentialCollected	This means that another Windows Hello companion device app has called the second API and that the Companion Authentication Service is verifying what was submitted. At this point, the Companion Authentication Service is not accepting any other authentication requests unless the currently submitted one does not pass verification. The Windows Hello companion device app should stay tuned until next state is reached.
CredentialAuthenticated	This means that the submitted credential worked. The credentialAuthenticated has the device ID of the Windows Hello companion device that succeeded. The Windows Hello companion device app should make sure to check on that to see if its associated device was the winner. If not, then the Windows Hello companion device app should avoid showing any post authentication flows (like success message on the companion device or perhaps a vibration on that device). Note that if the submitted credential did not work, the state will change to CollectingCredential state.
StoppingAuthentication	Authentication succeeded and user saw the desktop. Time to kill your background task. Before exiting the background task, explicitly unregister the StageEvent handler. This will help the background task exit quickly.

Windows Hello companion device apps should only call the two authentication APIs in the first two states. Windows Hello companion device apps should check for what scenario this event is being fired. There are two possibilities: unlock or post unlock. Currently, only unlock is supported. In upcoming releases, post unlock scenarios may be

supported. The SecondaryAuthenticationFactorAuthenticationScenario enum captures these two options:

C#

```
{
    SignIn = 0,           // Running under lock screen mode
    CredentialPrompt,     // Running post unlock
}
```

Complete code sample:

C#

```
using System;
using Windows.Security.Authentication.Identity.Provider;
using Windows.Storage.Streams;
using Windows.Security.Cryptography;
using System.Threading;
using Windows.ApplicationModel.Background;

namespace SecondaryAuthFactorSample
{
    public sealed class AuthenticationTask : IBackgroundTask
    {
        private string _deviceId;
        private static AutoResetEvent _exitTaskEvent = new
AutoResetEvent(false);
        private static IBackgroundTaskInstance _taskInstance;
        private BackgroundTaskDeferral _deferral;

        private void Authenticate()
        {
            int retryCount = 0;

            while (retryCount < 3)
            {
                //
                // Pseudo code, the svcAuthNonce should be passed to device
                // or generated from device
                //
                IBuffer svcAuthNonce =
CryptographicBuffer.GenerateRandom(256/8);

                SecondaryAuthenticationFactorAuthenticationResult authResult
= await
SecondaryAuthenticationFactorAuthentication.StartAuthenticationAsync(
                    _deviceId,
                    svcAuthNonce);
                if (authResult.Status !=
SecondaryAuthenticationFactorAuthenticationStatus.Started)
```

```

        {
            SecondaryAuthenticationFactorAuthenticationMessage
message;

            switch (authResult.Status)
            {
                case
SecondaryAuthenticationFactorAuthenticationStatus.DisabledByPolicy:
                    message =
SecondaryAuthenticationFactorAuthenticationMessage.DisabledByPolicy;
                    break;
                case
SecondaryAuthenticationFactorAuthenticationStatus.InvalidAuthenticationStage
:
                    // The task might need to wait for a
SecondaryAuthenticationFactorAuthenticationStageChangedEvent
                    break;
                default:
                    return;
            }

            // Show error message. Limited to 512 characters wide
await
SecondaryAuthenticationFactorAuthentication.ShowNotificationMessageAsync(nul
l, message);

            return;
        }

        //
        // Pseudo function:
        // The device calculates and returns sessionHmac and
deviceHmac

        //
await GetHmacsFromDevice(
    authResult.Authentication.ServiceAuthenticationHmac,
    authResult.Authentication.DeviceNonce,
    authResult.Authentication.SessionNonce,
    out deviceHmac,
    out sessionHmac);
if (sessionHmac == null ||
    deviceHmac == null)
{
    await
authResult.Authentication.AbortAuthenticationAsync(
    "Failed to read data from device");
    return;
}

        SecondaryAuthenticationFactorFinishAuthenticationStatus
status =

            await
authResult.Authentication.FinishAuthencationAsync(deviceHmac, sessionHmac);
            if (status ==
SecondaryAuthenticationFactorFinishAuthenticationStatus.NonceExpired)
            {
                retryCount++;
            }
        }
    }
}

```

```

        continue;
    }
    else if (status ==
SecondaryAuthenticationFactorFinishAuthenticationStatus.Completed)
    {
        // The credential data is collected and ready for unlock
        return;
    }
}

}

public void OnAuthenticationStageChanged(
    object sender,
    SecondaryAuthenticationFactorAuthenticationStageChangedEventArgs
args)
{
    // The application should check the args.StageInfo.Stage to
    determine what to do in next. Note that args.StageInfo.Scenario will have
    the scenario information (SignIn vs CredentialPrompt).

    switch(args.StageInfo.Stage)
    {
        case
SecondaryAuthenticationFactorAuthenticationStage.WaitingForUserConfirmation:
            // Show welcome message
            await
SecondaryAuthenticationFactorAuthentication.ShowNotificationMessageAsync(
                null,

SecondaryAuthenticationFactorAuthenticationMessage.WelcomeMessageSwipeUp);
            break;

        case
SecondaryAuthenticationFactorAuthenticationStage.CollectingCredential:
            // Authenticate device
            Authenticate();
            break;

        case
SecondaryAuthenticationFactorAuthenticationStage.CredentialAuthenticated:
            if (args.StageInfo.DeviceId == _deviceId)
            {
                // Show notification on device about PC unlock
            }
            break;

        case
SecondaryAuthenticationFactorAuthenticationStage.StoppingAuthentication:
            // Quit from background task
            _exitTaskEvent.Set();
            break;
    }

    Debug.WriteLine("Authentication Stage = " +
args.StageInfo.AuthenticationStage.ToString());
}

```

```

    }

    //
    // The Run method is the entry point of a background task.
    //
    public void Run(IBackgroundTaskInstance taskInstance)
    {
        _taskInstance = taskInstance;
        _deferral = taskInstance.GetDeferral();

        // Register canceled event for this task
        taskInstance.Canceled += TaskInstanceCanceled;

        // Find all device registered by this application
        IReadOnlyList<SecondaryAuthenticationFactorInfo> deviceInfoList
=
            await
SecondaryAuthenticationFactorRegistration.FindAllRegisteredDeviceInfoAsync(
            SecondaryAuthenticationFactorDeviceFindScope.AllUsers);

        if (deviceInfoList.Count == 0)
        {
            // Quit the task silently
            return;
        }
        _deviceId = deviceInfoList[0].DeviceId;
        Debug.WriteLine("Use first device '" + _deviceId + "' in the
list to signin");

        // Register AuthenticationStageChanged event

SecondaryAuthenticationFactorRegistration.AuthenticationStageChanged +=
OnAuthenticationStageChanged;

        // Wait the task exit event
        _exitTaskEvent.WaitOne();

        _deferral.Complete();
    }

    void TaskInstanceCanceled(IBackgroundTaskInstance sender,
BackgroundTaskCancellationReason reason)
    {
        _exitTaskEvent.Set();
    }
}

```

Register a background task

When the Windows Hello companion device app registers the first companion device, it should also register its background task component which will pass authentication

information between device and companion device authentication service.

C#

```
using System;
using Windows.Security.Authentication.Identity.Provider;
using Windows.Storage.Streams;
using Windows.ApplicationModel.Background;

namespace SecondaryAuthFactorSample
{
    public class BackgroundTaskManager
    {
        // Register background task
        public static async Task<IBackgroundTaskRegistration>
        GetOrRegisterBackgroundTaskAsync(
            string bgTaskName,
            string taskEntryPoint)
        {
            // Check if there's an existing background task already
            registered
            var bgTask = (from t in BackgroundTaskRegistration.AllTasks
                          where t.Value.Name.Equals(bgTaskName)
                          select t.Value).SingleOrDefault();
            if (bgTask == null)
            {
                BackgroundAccessStatus status =

                BackgroundExecutionManager.RequestAccessAsync().AsTask().GetAwaiter().GetResult();

                if (status == BackgroundAccessStatus.Denied)
                {
                    Debug.WriteLine("Background Execution is denied.");
                    return null;
                }

                var taskBuilder = new BackgroundTaskBuilder();
                taskBuilder.Name = bgTaskName;
                taskBuilder.TaskEntryPoint = taskEntryPoint;
                taskBuilder.SetTrigger(new
                SecondaryAuthenticationFactorAuthenticationTrigger());
                bgTask = taskBuilder.Register();
                // Background task is registered
            }

            bgTask.Completed += BgTask_Completed;
            bgTask.Progress += BgTask_Progress;

            return bgTask;
        }
    }
}
```

Errors and messages

The Windows Hello companion device framework is responsible for providing feedback to the user about success or failure of signing in. The Windows Hello companion device framework will provide a stock of (localized) text and error messages for the Windows Hello companion device app to choose from. These will be displayed in the logon UI.



Windows Hello companion device apps can use `ShowNotificationMessageAsync` to show messages to user as part of the logon UI. Call this API when an intent signal is available. Note that an intent signal must always be collected on the Windows Hello companion device side.

There are two types of messages: guidance and errors.

Guidance messages are designed to show the user how to start the unlock process. These messages are only shown to the user once on the lock screen, upon first device registration, and never shown there again. These messages will continue to be shown under the lock screen.

Error messages are always shown and will be shown after an intent signal is provided. Given that an intent signal must be collected before showing messages to the user, and the user will provide that intent only using one of the Windows Hello companion devices, there must not be a situation where multiple Windows Hello companion devices race for showing error messages. As a result, the Windows Hello companion device framework does not maintain any queue. When a caller asks for an error message, it will be shown for 5 seconds and all other requests for showing an error

message in that 5 seconds are dropped. Once 5 seconds has passed, then the opportunity arises for another caller to show an error message. We prohibit any caller from jamming the error channel.

Guidance and error messages are as follows. Device name is a parameter passed by the companion device app as part of `ShowNotificationMessageAsync`.

Guidance

- "Swipe up or press space bar to sign in with *device name*."
- "Setting up your companion device. Please wait or use another sign-in option."
- "Tap *device name* to the NFC reader to sign in."
- "Looking for *device name* ..."
- "Plug *device name* into a USB port to sign in."

Errors

- "See *device name* for sign-in instructions."
- "Turn on Bluetooth to use *device name* to sign in."
- "Turn on NFC to use *device name* to sign in."
- "Connect to a Wi-Fi network to use *device name* to sign in."
- "Tap *device name* again."
- "Your enterprise prevents sign in with *device name*. Use another sign-in option."
- "Tap *device name* to sign in."
- "Rest your finger on *device name* to sign in."
- "Swipe your finger on *device name* to sign in."
- "Couldn't sign in with *device name*. Use another sign-in option."
- "Something went wrong. Use another sign-in option, and then set up *device name* again."
- "Try again."
- "Say your Spoken Passphrase into *device name*."
- "Ready to sign in with *device name*."
- "Use another sign-in option first, then you can use *device name* to sign in."

Enumerating registered devices

The Windows Hello companion device app can enumerate the list of registered companion devices via `FindAllRegisteredDeviceInfoAsync` call. This API supports two query types defined via enum `SecondaryAuthenticationFactorDeviceFindScope`:

C#

```
{  
    User = 0,  
    AllUsers,  
}
```

The first scope returns the list of devices for the logged on user. The second one returns the list for all users on that PC. The first scope must be used at un-registration time to avoid un-registering another user's Windows Hello companion device. The second one must be used at authentication or registration time: at registration time, this enumeration can help the app avoid trying to register the same Windows Hello companion device twice.

Note that even if the app does not perform this check, the PC does and will reject the same Windows Hello companion device from being registered more than once. At authentication time, using the AllUsers scope helps the Windows Hello companion device app support switch user flow: log on user A when user B is logged in (this requires that both users have installed the Windows Hello companion device app and user A has registered their companion devices with the PC and the PC is sitting on lock screen (or logon screen)).

Security requirements

The Companion Authentication Service provides the following security protections.

- Malware on a Windows 10 desktop device running as a medium user or app container cannot use the Windows Hello companion device to access user credential keys (stored as part of Windows Hello) on a PC silently.
- A malicious user on a Windows 10 desktop device cannot use the Windows Hello companion device that belongs to another user on that Windows 10 desktop device to get silent access to his user credential keys (on the same Windows 10 desktop device).
- Malware on the Windows Hello companion device cannot silently get access to user credential keys on a Windows 10 desktop device, including leveraging functionality or code developed specifically for the Windows Hello companion device framework.
- A malicious user cannot unlock a Windows 10 desktop device by capturing traffic between the Windows Hello companion device and the Windows 10 desktop device and replaying it later. Usage of nonce, authkey, and HMAC in our protocol guarantees protection against a replay attack.
- Malware or a malicious user on a rouge PC cannot use Windows Hello companion device to get access to honest user PC. This is achieved through mutual

authentication between Companion Authentication Service and Windows Hello companion device through usage of authkey and HMAC in our protocol.

The key to achieve the security protections enumerated above is to protect HMAC keys from unauthorized access and also verifying user presence. More specifically, it must satisfy these requirements:

- Provide protection against cloning the Windows Hello companion device
- Provide protection against eavesdropping when sending HMAC keys at registration time to the PC
- Make sure that user presence signal is available

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Web authentication broker

Article • 10/20/2022

This article explains how to connect your Universal Windows Platform (UWP) app to an online identity provider that uses authentication protocols like OpenID or OAuth. The [AuthenticateAsync](#) method sends a request to the online identity provider and gets back an access token that describes the provider resources to which the app has access.

⚠ Note

For a complete, working code sample, clone the [WebAuthenticationBroker repo on GitHub](#).

Register your app with your online provider

You must register your app with the online identity provider to which you want to connect. You can find out how to register your app from the identity provider. After registering, the online provider typically gives you an Id or secret key for your app.

Build the authentication request URI

The request URI consists of the address where you send the authentication request to your online provider appended with other required information, such as an app ID or secret, a redirect URI where the user is sent after completing authentication, and the expected response type. You can find out from your provider what parameters are required.

The request URI is sent as the *requestUri* parameter of the [AuthenticateAsync](#) method. It must be a secure address (it must start with `https://`)

The following example shows how to build the request URI.

CS

```
string startURL = "https://<providerendpoint>?client_id=<clientid>&scope=<scopes>&response_type=token";  
string endURL = "http://<appendpoint>";
```

```
System.Uri startURI = new System.Uri(startURL);
System.Uri endURI = new System.Uri(endURL);
```

Connect to the online provider

You call the `AuthenticateAsync` method to connect to the online identity provider and get an access token. The method takes the URI constructed in the previous step as the *requestUri* parameter, and a URI to which you want the user to be redirected as the *callbackUri* parameter.

cs

```
string result;

try
{
    var webAuthenticationResult =
        await
Windows.Security.Authentication.Web.WebAuthenticationBroker.AuthenticateAsyn
c(
    Windows.Security.Authentication.Web.WebAuthenticationOptions.None,
    startURI,
    endURI);

    switch (webAuthenticationResult.ResponseStatus)
    {
        case
Windows.Security.Authentication.Web.WebAuthenticationStatus.Success:
            // Successful authentication.
            result = webAuthenticationResult.ResponseData.ToString();
            break;
        case
Windows.Security.Authentication.Web.WebAuthenticationStatus.ErrorHttp:
            // HTTP error.
            result = webAuthenticationResult.ResponseErrorDetail.ToString();
            break;
        default:
            // Other error.
            result = webAuthenticationResult.ResponseData.ToString();
            break;
    }
}
catch (Exception ex)
{
    // Authentication failed. Handle parameter, SSL/TLS, and Network
    Unavailable errors here.
    result = ex.Message;
}
```

Warning

In addition to [AuthenticateAsync](#), the [Windows.Security.Authentication.Web](#) namespace contains an [AuthenticateAndContinue](#) method. Do not call this method. It is designed for apps targeting Windows Phone 8.1 only and is deprecated starting with Windows 10.

Connecting with single sign-on (SSO).

By default, Web authentication broker does not allow cookies to persist. Because of this, even if the app user indicates that they want to stay logged in (for example, by selecting a check box in the provider's login dialog), they will have to login each time they want to access resources for that provider. To login with SSO, your online identity provider must have enabled SSO for Web authentication broker, and your app must call the overload of [AuthenticateAsync](#) that does not take a *callbackUri* parameter. This will allow persisted cookies to be stored by the web authentication broker, so that future authentication calls by the same app will not require repeated sign-in by the user (the user is effectively "logged in" until the access token expires).

To support SSO, the online provider must allow you to register a redirect URI in the form `ms-app://<appSID>`, where `<appSID>` is the SID for your app. You can find your app's SID from the app developer page for your app, or by calling the [GetCurrentApplicationCallbackUri](#) method.

CS

```
string result;

try
{
    var webAuthenticationResult =
        await
Windows.Security.Authentication.Web.WebAuthenticationBroker.AuthenticateAsyn
c(
    Windows.Security.Authentication.Web.WebAuthenticationOptions.None,
    startURI);

    switch (webAuthenticationResult.ResponseStatus)
    {
        case
Windows.Security.Authentication.Web.WebAuthenticationStatus.Success:
            // Successful authentication.
            result = webAuthenticationResult.ResponseData.ToString();
            break;
        case
```



```

Windows.Security.Authentication.Web.WebAuthenticationStatus.ErrorHttp:
    // HTTP error.
    result = webAuthenticationResult.ResponseErrorDetail.ToString();
    break;
default:
    // Other error.
    result = webAuthenticationResult.ResponseData.ToString();
    break;
}
}
catch (Exception ex)
{
    // Authentication failed. Handle parameter, SSL/TLS, and Network
    // Unavailable errors here.
    result = ex.Message;
}

```

Debugging

There are several ways to troubleshoot the web authentication broker APIs, including reviewing operational logs and reviewing web requests and responses using Fiddler.

Operational logs

Often you can determine what is not working by using the operational logs. There is a dedicated event log channel Microsoft-Windows-WebAuth\Operational that allows website developers to understand how their web pages are being processed by the Web authentication broker. To enable it, launch eventvwr.exe and enable Operational log under the Application and Services\Microsoft\Windows\WebAuth. Also, the Web authentication broker appends a unique string to the user agent string to identify itself on the web server. The string is "MSAuthHost/1.0". Note that the version number may change in the future, so you should not depend on that version number in your code. An example of the full user agent string, followed by full debugging steps, is as follows.

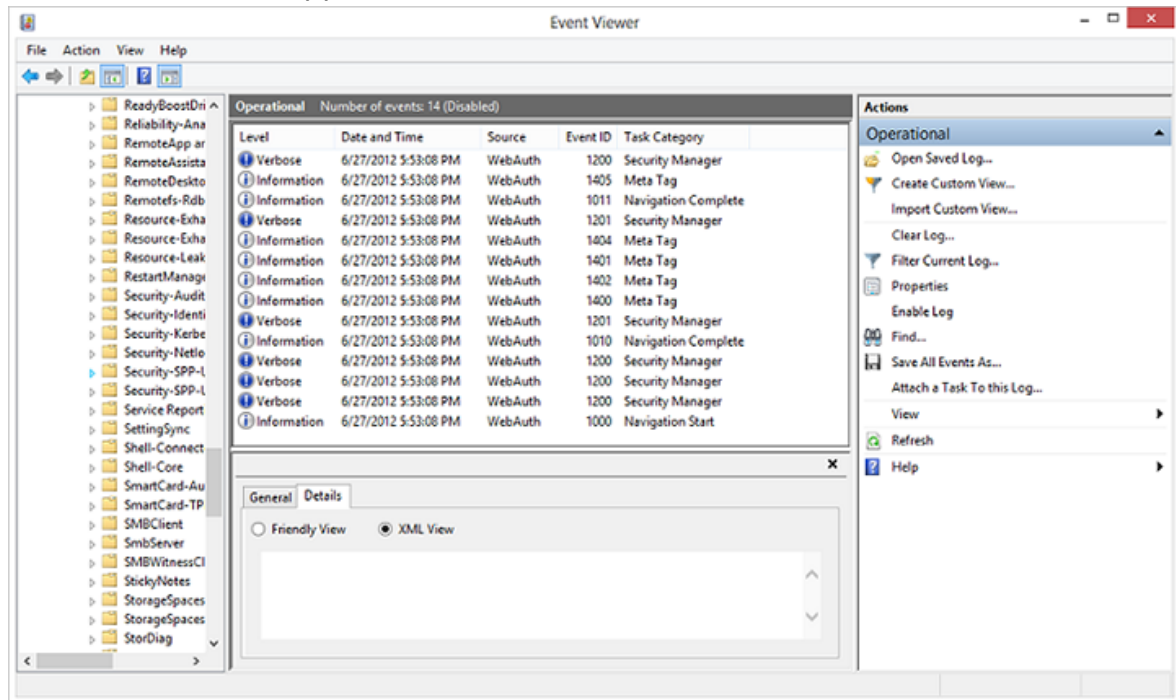
```

User-Agent: Mozilla/5.0 (compatible; MSIE 10.0; Windows NT 6.2; Win64; x64;
Trident/6.0; MSAuthHost/1.0)

```

1. Enable operational logs.

2. Run Contoso social application.



3. The generated logs entries can be used to understand the behavior of Web authentication broker in greater detail. In this case, these can include:

- Navigation Start: Logs when the AuthHost is started and contains information about the start and termination URLs.



- Navigation Complete: Logs the completion of loading a web page.
- Meta Tag: Logs when a meta-tag is encountered including the details.
- Navigation Terminate: Navigation terminated by the user.
- Navigation Error: AuthHost encounters a navigation error at a URL including HttpStatusCode.
- Navigation End: Terminating URL is encountered.

Fiddler

The Fiddler web debugger can be used with apps. For more information, see [Fiddler documentation](#)

1. Since the AuthHost runs in its own app container, to give it the private network capability you must set a registry key: Windows Registry Editor Version 5.00

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows  
NT\CurrentVersion\Image File Execution  
Options\authhost.exe\EnablePrivateNetwork = 00000001
```

If you do not have this registry key, you can create it in a Command Prompt with administrator privileges.

Windows Command Prompt

```
REG ADD "HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File  
Execution Options\authhost.exe" /v EnablePrivateNetwork /t REG_DWORD /d  
1 /f
```

2. Add a rule for the AuthHost as this is what is generating the outbound traffic.

syntax

```
CheckNetIsolation.exe LoopbackExempt -a -  
n=microsoft.windows.authhost.a.p_8wekyb3d8bbwe  
CheckNetIsolation.exe LoopbackExempt -a -  
n=microsoft.windows.authhost.sso.p_8wekyb3d8bbwe  
CheckNetIsolation.exe LoopbackExempt -a -  
n=microsoft.windows.authhost.sso.c_8wekyb3d8bbwe  
D:\Windows\System32>CheckNetIsolation.exe LoopbackExempt -s  
List Loopback Exempted AppContainers  
[1] -----  
    Name: microsoft.windows.authhost.sso.c_8wekyb3d8bbwe  
    SID:  S-1-15-2-1973105767-3975693666-32999980-3747492175-  
1074076486-3102532000-500629349  
[2] -----  
    Name: microsoft.windows.authhost.sso.p_8wekyb3d8bbwe  
    SID:  S-1-15-2-166260-4150837609-3669066492-3071230600-3743290616-  
3683681078-2492089544  
[3] -----  
    Name: microsoft.windows.authhost.a.p_8wekyb3d8bbwe  
    SID:  S-1-15-2-3506084497-1208594716-3384433646-2514033508-  
1838198150-1980605558-3480344935
```

3. Add a firewall rule for incoming traffic to Fiddler.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Cryptography

Article • 10/20/2022

The article provides an overview of the cryptography features available to Universal Windows Platform (UWP) apps. For detailed information on particular tasks, see the table at the end of this article.

Terminology

The following terminology is commonly used in cryptography and public key infrastructure (PKI).

Term	Description
Encryption	The process of transforming data by using a cryptographic algorithm and key. The transformed data can be recovered only by using the same algorithm and the same (symmetric) or related (public) key.
Decryption	The process of returning encrypted data to its original form.
Plaintext	Originally referred to an unencrypted text message. Currently refers to any unencrypted data.
Ciphertext	Originally referred to an encrypted, and therefore unreadable, text message. Currently refers to any encrypted data.
Hashing	The process of converting variable length data into a fixed length, typically smaller, value. By comparing hashes, you can obtain reasonable assurance that two or more data are the same.
Signature	Encrypted hash of digital data typically used to authenticate the sender of the data or verify that the data was not tampered with during transmission.
Algorithm	A step-by-step procedure for encrypting data.
Key	A random or pseudorandom number used as input to a cryptographic algorithm to encrypt and decrypt data.
Symmetric Key Cryptography	Cryptography in which encryption and decryption use the same key. This is also known as secret key cryptography.
Asymmetric Key Cryptography	Cryptography in which encryption and decryption use a different but mathematically related key. This is also called public key cryptography.

Term	Description
Encoding	The process of encoding digital messages, including certificates, for transport across a network.
Algorithm Provider	A DLL that implements a cryptographic algorithm.
Key Storage Provider	A container for storing key material. Currently, keys can be stored in software, smart cards, or the trusted platform module (TPM).
X.509 Certificate	A digital document, typically issued by a certification authority, to verify the identity of an individual, system, or entity to other interested parties.

Namespaces

The following namespaces are available for use in apps.

Windows.Security.Cryptography

Contains the `CryptographicBuffer` class and static methods that enable you to:

- Convert data to and from strings
- Convert data to and from byte arrays
- Encode messages for network transport
- Decode messages after transport

Windows.Security.Cryptography.Certificates

Contains classes, interfaces, and enumeration types that enable you to:

- Create a certificate request
- Install a certificate response
- Import a certificate in a PFX file
- Specify and retrieve certificate request properties

Windows.Security.Cryptography.Core

Contains classes and enumeration types that enable you to:

- Encrypt and decrypt data
- Hash data

- Sign data and verify signatures
- Create, import, and export keys
- Work with asymmetric key algorithm providers
- Work with symmetric key algorithm providers
- Work with hash algorithm providers
- Work with machine authentication code (MAC) algorithm providers
- Work with key derivation algorithm providers

Windows.Security.Cryptography.DataProtection

Contains classes that enable you to:

- Asynchronously encrypt and decrypt static data
- Asynchronously encrypt and decrypt data streams

Crypto and PKI application capabilities

The simplified application programming interface available for apps enables the following cryptographic and public key infrastructure (PKI) capabilities.

Cryptography support

You can perform the following cryptographic tasks. For more information, see the [Windows.Security.Cryptography.Core](#) namespace.

- Create symmetric keys
- Perform symmetric encryption
- Create asymmetric keys
- Perform asymmetric encryption
- Derive password based keys
- Create message authentication codes (MACs)
- Hash content
- Digitally sign content

The SDK also provides a simplified interface for password-based data protection. You can use this to perform the following tasks. For more information, see the [Windows.Security.Cryptography.DataProtection](#) namespace.

- Asynchronous protection of static data
- Asynchronous protection of a data stream

Encoding support

An app can encode cryptographic data for transmission across a network and decode data received from a network source. For more information, see the static methods available in the [Windows.Security.Cryptography](#) namespace.

PKI support

Apps can perform the following PKI tasks. For more information, see the [Windows.Security.Cryptography.Certificates](#) namespace.

- Create a certificate
- Create a self-signed certificate
- Install a certificate response
- Import a certificate in PFX format
- Use smart card certificates and keys (sharedUserCertificates capabilities set)
- Use certificates from the user MY store (sharedUserCertificates capabilities set)

Additionally, you can use the manifest to perform the following actions:

- Specify per application trusted root certificates
- Specify per application peer trusted certificates
- Explicitly disable inheritance from system trust
- Specify the certificate selection criteria
 - Hardware certificates only
 - Certificates that chain through a specified set of issuers
 - Automatically select a certificate from the application store

Detailed articles

The following articles provide more detail on security scenarios:

Topic	Description
Certificates	This article discusses the use of certificates in UWP apps. Digital certificates are used in public key cryptography to bind a public key to a person, computer, or organization. The bound identities are most often used to authenticate one entity to another. For example, certificates are often used to authenticate a web server to a user and a user to a web server. You can create certificate requests and install or import issued certificates. You can also enroll a certificate in a certificate hierarchy.

Topic	Description
Cryptographic keys	This article shows how to use standard key derivation functions to derive keys and how to encrypt content using symmetric and asymmetric keys.
Data protection	This article explains how to use the DataProtectionProvider class in the Windows.Security.Cryptography.DataProtection namespace to encrypt and decrypt digital data in a UWP app.
MACs, hashes, and signatures	This article discusses how message authentication codes (MACs), hashes, and signatures can be used in UWP apps to detect message tampering.
Export restrictions on cryptography	Use this info to determine if your app uses cryptography in a way that might prevent it from being listed in the Microsoft Store.
Common cryptography tasks	These articles provide example code for common UWP cryptography tasks, such as creating random numbers, comparing buffers, converting between strings and binary data, copying to and from byte arrays, and encoding and decoding data.

Intro to certificates

Article • 10/20/2022

This article discusses the use of certificates in Universal Windows Platform (UWP) apps. Digital certificates are used in public key cryptography to bind a public key to a person, computer, or organization. The bound identities are most often used to authenticate one entity to another. For example, certificates are often used to authenticate a web server to a user and a user to a web server. You can create certificate requests and install or import issued certificates. You can also enroll a certificate in a certificate hierarchy.

Shared certificate stores

UWP apps use the new isolationist application model introduced in Windows 8. In this model, apps run in low-level operating system construct, called an app container, that prohibits the app from accessing resources or files outside of itself unless explicitly permitted to do so. The following sections describe the implications this has on public key infrastructure (PKI).

Certificate storage per app container

Certificates that are intended for use in a specific app container are stored in per user, per app container locations. An app running in an app container has write access to only its own certificate storage. If the application adds certificates to any of its stores, these certificates cannot be read by other apps. If an app is uninstalled, any certificates specific to it are also removed. An app also has read access to local machine certificate stores other than the MY and REQUEST store.

Cache

Each app container has an isolated cache in which it can store issuer certificates needed for validation, certificate revocation lists (CRL), and online certificate status protocol (OCSP) responses.

Shared certificates and keys

When a smart card is inserted into a reader, the certificates and keys contained on the card are propagated to the user MY store where they can be shared by any full-trust application the user is running. By default, however, app containers do not have access to the per user MY store.

To address this issue and enable groups of principals to access groups of resources, the app container isolation model supports the capabilities concept. A capability allows an app container process to access a specific resource. The `sharedUserCertificates` capability grants an app container read access to the certificates and keys contained in the user MY store and the Smart Card Trusted Roots store. The capability does not grant read access to the user REQUEST store.

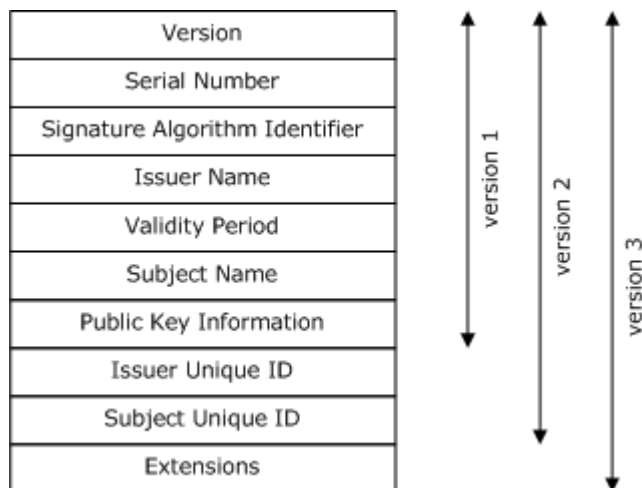
You specify the `sharedUserCertificates` capability in the manifest as shown in the following example.

XML

```
<Capabilities>
  <Capability Name="sharedUserCertificates" />
</Capabilities>
```

Certificate fields

The X.509 public key certificate standard has been revised over time. Each successive version of the data structure has retained the fields that existed in the previous versions and added more, as shown in the following illustration.



Some of these fields and extensions can be specified directly when you use the [CertificateRequestProperties](#) class to create a certificate request. Most cannot. These fields can be filled by the issuing authority or they can be left blank. For more information about the fields, see the following sections:

Version 1 fields

Field	Description
-------	-------------

Field	Description
Version	Specifies the version number of the encoded certificate. Currently, the possible values of this field are 0, 1, or 2.
Serial Number	Contains a positive, unique integer assigned by the certification authority (CA) to the certificate.
Signature Algorithm	Contains an object identifier (OID) that specifies the algorithm used by the CA to sign the certificate. For example, 1.2.840.113549.1.1.5 specifies a SHA-1 hashing algorithm combined with the RSA encryption algorithm from RSA Laboratories.
Issuer	Contains the X.500 distinguished name (DN) of the CA that created and signed the certificate.
Validity	Specifies the time interval during which the certificate is valid. Dates through the end of 2049 use the Coordinated Universal Time (Greenwich Mean Time) format (yymmddhhmmssz). Dates beginning with January 1st, 2050 use the generalized time format (yyyymmddhhmmssz).
Subject	Contains an X.500 distinguished name of the entity associated with the public key contained in the certificate.
Public Key	Contains the public key and associated algorithm information.

Version 2 fields

An X.509 version 2 certificate contains the basic fields defined in version 1 and adds the following fields.

Field	Description
Issuer Unique Identifier	Contains a unique value that can be used to make the X.500 name of the CA unambiguous when reused by different entities over time.
Subject Unique Identifier	Contains a unique value that can be used to make the X.500 name of the certificate subject unambiguous when reused by different entities over time.

Version 3 extensions

An X.509 version 3 certificate contains the fields defined in version 1 and version 2 and adds certificate extensions.

Field	Description
-------	-------------

Field	Description
Authority Key Identifier	Identifies the certification authority (CA) public key that corresponds to the CA private key used to sign the certificate.
Basic Constraints	Specifies whether the entity can be used as a CA and, if so, the number of subordinate CAs that can exist beneath it in the certificate chain.
Certificate Policies	Specifies the policies under which the certificate has been issued and the purposes for which it can be used.
CRL Distribution Points	Contains the URI of the base certificate revocation list (CRL).
Enhanced Key Usage	Specifies the manner in which the public key contained in the certificate can be used.
Issuer Alternative Name	Specifies one or more alternative name forms for the issuer of the certificate request.
Key Usage	Specifies restrictions on the operations that can be performed by the public key contained in the certificate.
Name Constraints	Specifies the namespace within which all subject names in a certificate hierarchy must be located. The extension is used only in a CA certificate.
Policy Constraints	Constrains path validation by prohibiting policy mapping or by requiring that each certificate in the hierarchy contain an acceptable policy identifier. The extension is used only in a CA certificate.
Policy Mappings	Specifies the policies in a subordinate CA that correspond to policies in the issuing CA.
Private Key Usage Period	Specifies a different validity period for the private key than for the certificate with which the private key is associated.
Subject Alternative Name	Specifies one or more alternative name forms for the subject of the certificate request. Example alternative forms include email addresses, DNS names, IP addresses, and URIs.
Subject Directory Attributes	Conveys identification attributes such as the nationality of the certificate subject. The extension value is a sequence of OID-value pairs.
Subject Key Identifier	Differentiates between multiple public keys held by the certificate subject. The extension value is typically a SHA-1 hash of the key.

Cryptographic keys

Article • 10/20/2022

This article shows how to use standard key derivation functions to derive keys and how to encrypt content using symmetric and asymmetric keys.

Symmetric keys

Symmetric key encryption, also called secret key encryption, requires that the key used for encryption also be used for decryption. You can use a [SymmetricKeyAlgorithmProvider](#) class to specify a symmetric algorithm and create or import a key. You can use static methods on the [CryptographicEngine](#) class to encrypt and decrypt data by using the algorithm and key.

Symmetric key encryption typically uses block ciphers and block cipher modes. A block cipher is a symmetric encryption function that operates on fixed size blocks. If the message you want to encrypt is longer than the block length, you must use a block cipher mode. A block cipher mode is a symmetric encryption function built by using a block cipher. It encrypts plaintext as a series of fixed size blocks. The following modes are supported for apps:

- The ECB (electronic codebook) mode encrypts each block of the message separately. This is not considered a secure encryption mode.
- The CBC (cipher block chaining) mode uses the previous ciphertext block to obfuscate the current block. You must determine what value to use for the first block. This value is called the initialization vector (IV).
- The CCM (counter with CBC-MAC) mode combines the CBC block cipher mode with a message authentication code (MAC).
- The GCM (Galois counter mode) mode combines the counter encryption mode with the Galois authentication mode.

Some modes such as CBC require that you use an initialization vector (IV) for the first ciphertext block. The following are common initialization vectors. You specify the IV when calling [CryptographicEngine.Encrypt](#). For most cases it is important that the IV never be reused with the same key.

- Fixed uses the same IV for all messages to be encrypted. This leaks information and its use is not recommended.
- Counter increments the IV for each block.
- Random creates a pseudorandom IV. You can use [CryptographicBuffer.GenerateRandom](#) to create the IV.

- Nonce-Generated uses a unique number for each message to be encrypted. Typically, the nonce is a modified message or transaction identifier. The nonce does not have to be kept secret, but it should never be reused under the same key.

Most modes require that the length of the plaintext be an exact multiple of the block size. This usually requires that you pad the plaintext to obtain the appropriate length.

While block ciphers encrypt fixed size blocks of data, stream ciphers are symmetric encryption functions that combine plaintext bits with a pseudorandom bit stream (called a key stream) to generate the ciphertext. Some block cipher modes such as output feedback mode (OTF) and counter mode (CTR) effectively turn a block cipher into a stream cipher. Actual stream ciphers such as RC4, however, typically operate at higher speeds than block cipher modes are capable of achieving.

The following example shows how to use the [SymmetricKeyAlgorithmProvider](#) class to create a symmetric key and use it to encrypt and decrypt data.

Asymmetric keys

Asymmetric key cryptography, also called public key cryptography, uses a public key and a private key to perform encryption and decryption. The keys are different but mathematically related. Typically the private key is kept secret and is used to decrypt data while the public key is distributed to interested parties and is used to encrypt data. Asymmetric cryptography is also useful for signing data.

Because asymmetric cryptography is much slower than symmetric cryptography, it is seldom used to encrypt large amounts of data directly. Instead, it is typically used in the following manner to encrypt keys.

- Alice requires that Bob send her only encrypted messages.
- Alice creates a private/public key pair, keeps her private key secret and publishes her public key.
- Bob has a message he wants to send to Alice.
- Bob creates a symmetric key.
- Bob uses his new symmetric key to encrypt his message to Alice.
- Bob uses Alice's public key to encrypt his symmetric key.
- Bob sends the encrypted message and the encrypted symmetric key to Alice (enveloped).
- Alice uses her private key (from the private/public pair) to decrypt Bob's symmetric key.
- Alice uses Bob's symmetric key to decrypt the message.

You can use an [AsymmetricKeyAlgorithmProvider](#) object to specify an asymmetric algorithm or a signing algorithm, to create or import an ephemeral key pair, or to import the public key portion of a key pair.

Deriving keys

It is often necessary to derive additional keys from a shared secret. You can use the [KeyDerivationAlgorithmProvider](#) class and one of the following specialized methods in the [KeyDerivationParameters](#) class to derive keys.

Object	Description
BuildForPbkdf2	Creates a KeyDerivationParameters object for use in the password-based key derivation function 2 (PBKDF2).
BuildForSP800108	Creates a KeyDerivationParameters object for use in a counter mode, hash-based message authentication code (HMAC) key derivation function.
BuildForSP80056a	Creates a KeyDerivationParameters object for use in the SP800-56A key derivation function.

Data protection

Article • 10/20/2022

This article explains how to use the [DataProtectionProvider](#) class in the [Windows.Security.Cryptography.DataProtection](#) namespace to encrypt and decrypt digital data in a UWP app.

You can use the data protection APIs in multiple ways:

- To protect data to an Active Directory (AD) security principal like an AD group. Any member of the group can decrypt the data.
- To protect data to the public key contained in an X.509 certificate. The owner of the private key can decrypt the data.
- To protect data by using a symmetric key. This works, for example, to protect data to a non-AD principal such as Live ID.
- To protect data to the credentials (password) used during logon to a website.

To protect data, when you create a [DataProtectionProvider](#) object you must specify a protection descriptor before calling [ProtectAsync](#) or [ProtectStreamAsync](#). The following example shows possible sample protection descriptors.

Protecting static data

The following example shows how to use the [ProtectAsync](#) and [UnprotectAsync](#) methods to asynchronously protect static data to the current user's SID.

C#

```
using Windows.Security.Cryptography;
using Windows.Security.Cryptography.DataProtection;
using Windows.Storage.Streams;
using System.Threading.Tasks;

namespace SampleProtectAsync
{
    sealed partial class StaticDataProtectionApp : Application
    {
        public StaticDataProtectionApp()
        {
            // Initialize the application.
            this.InitializeComponent();

            // Protect data asynchronously.
            this.Protect();
        }
    }
}
```



```

public async void Protect()
{
    // Initialize function arguments.
    String strMsg = "This is a message to be protected.";
    String strDescriptor = "LOCAL=user";
    BinaryStringEncoding encoding = BinaryStringEncoding.Utf8;

    // Protect a message to the local user.
    IBuffer buffProtected = await this.SampleProtectAsync(
        strMsg,
        strDescriptor,
        encoding);

    // Decrypt the previously protected message.
    String strDecrypted = await this.SampleUnprotectData(
        buffProtected,
        encoding);
}

public async Task<IBuffer> SampleProtectAsync(
    String strMsg,
    String strDescriptor,
    BinaryStringEncoding encoding)
{
    // Create a DataProtectionProvider object for the specified
descriptor.
    DataProtectionProvider Provider = new
DataProtectionProvider(strDescriptor);

    // Encode the plaintext input message to a buffer.
    encoding = BinaryStringEncoding.Utf8;
    IBuffer buffMsg =
CryptographicBuffer.ConvertStringToBinary(strMsg, encoding);

    // Encrypt the message.
    IBuffer buffProtected = await Provider.ProtectAsync(buffMsg);

    // Execution of the SampleProtectAsync function resumes here
// after the awaited task (Provider.ProtectAsync) completes.
    return buffProtected;
}

public async Task<String> SampleUnprotectData(
    IBuffer buffProtected,
    BinaryStringEncoding encoding)
{
    // Create a DataProtectionProvider object.
    DataProtectionProvider Provider = new DataProtectionProvider();

    // Decrypt the protected message specified on input.
    IBuffer buffUnprotected = await
Provider.UnprotectAsync(buffProtected);

    // Execution of the SampleUnprotectData method resumes here

```

```

        // after the awaited task (Provider.UnprotectAsync) completes
        // Convert the unprotected message from an IBuffer object to a
string.
        String strClearText =
CryptographicBuffer.ConvertBinaryToString(encoding, buffUnprotected);

        // Return the plaintext string.
        return strClearText;
    }
}
}

```

Protecting stream data

The following example shows how to use the [ProtectStreamAsync](#) and [UnprotectStreamAsync](#) methods to asynchronously protect stream data to the current user's SID.

C#

```

using Windows.Security.Cryptography;
using Windows.Security.Cryptography.DataProtection;
using Windows.Storage.Streams;
using System.Threading.Tasks;

namespace SampleProtectStreamAsync
{
    sealed partial class StreamDataProtectionApp : Application
    {
        public StreamDataProtectionApp()
        {
            // Initialize the application.
            this.InitializeComponent();

            // Protect a stream synchronously
            this.ProtectData();
        }

        public async void ProtectData()
        {
            // Initialize function arguments.
            String strDescriptor = "LOCAL=user";
            String strLoremIpsum = "Lorem ipsum dolor sit amet, consectetur
adipiscing elit. Suspendisse elementum "
                + "ullamcorper eros, vitae gravida nunc consequat
sollicitudin. Vivamus lacinia, "
                + "diam a molestie porttitor, sapien neque volutpat est, non
suscipit leo dolor "
                + "sit amet nisl. Praesent tincidunt tincidunt quam ut
pharetra. Sed tincidunt "

```

```

        + "sit amet nisl. Praesent tincidunt tincidunt quam ut
pharetra. Sed tincidunt "
        + "porttitor massa, at convallis dolor dictum suscipit.
Nullam vitae lectus in "
        + "lorem scelerisque convallis sed scelerisque orci.
Praesent sed ligula vel erat "
        + "eleifend tempus. Nullam dignissim aliquet mauris a
aliquet. Nulla augue justo, "
        + "posuere a consectetur ut, suscipit et sem. Proin eu
libero ut felis tincidunt "
        + "interdum. Curabitur vulputate eros nec sapien elementum
ut dapibus eros "
        + "dapibus. Suspendisse quis dui dolor, non imperdiet leo.
In consequat, odio nec "
        + "aliquam tincidunt, magna enim ultrices massa, ac pharetra
est urna at arcu. "
        + "Nunc suscipit, velit non interdum suscipit, lectus lectus
auctor tortor, quis "
        + "ultrices orci felis in dolor. Etiam congue pretium libero
eu vestibulum. "
        + "Mauris bibendum erat eleifend nibh consequat eu pharetra
metus convallis. "
        + "Morbi sem eros, venenatis vel vestibulum consequat,
hendrerit rhoncus purus.";

```

```

    BinaryStringEncoding encoding = BinaryStringEncoding.Utf16BE;

```

```

    // Encrypt the data as a stream.

```

```

    IBuffer buffProtected = await this.SampleDataProtectionStream(
        strDescriptor,
        strLoremIpsum,
        encoding);

```

```

    // Decrypt a data stream.

```

```

    String strUnprotected = await this.SampleDataUnprotectStream(
        buffProtected,
        encoding);

```

```

}

```

```

public async Task<IBuffer> SampleDataProtectionStream(
    String descriptor,
    String strMsg,
    BinaryStringEncoding encoding)
{

```

```

    // Create a DataProtectionProvider object for the specified
descriptor.

```

```

    DataProtectionProvider Provider = new
DataProtectionProvider(descriptor);

```

```

    // Convert the input string to a buffer.

```

```

    IBuffer buffMsg =
CryptographicBuffer.ConvertStringToBinary(strMsg, encoding);

```

```

    // Create a random access stream to contain the plaintext
message.

```

```

    InMemoryRandomAccessStream inputData = new

```

```

InMemoryRandomAccessStream();

        // Create a random access stream to contain the encrypted
message.
        InMemoryRandomAccessStream protectedData = new
InMemoryRandomAccessStream();

        // Retrieve an IOOutputStream object and fill it with the input
(plaintext) data.
        IOOutputStream outputStream = inputData.GetOutputStreamAt(0);
        DataWriter writer = new DataWriter(outputStream);
        writer.WriteBuffer(buffMsg);
        await writer.StoreAsync();
        await outputStream.FlushAsync();

        // Retrieve an IInputStream object from which you can read the
input data.
        IInputStream source = inputData.GetInputStreamAt(0);

        // Retrieve an IOOutputStream object and fill it with encrypted
data.
        IOOutputStream dest = protectedData.GetOutputStreamAt(0);
        await Provider.ProtectStreamAsync(source, dest);
        await dest.FlushAsync();

        //Verify that the protected data does not match the original
        DataReader reader1 = new
DataReader(inputData.GetInputStreamAt(0));
        DataReader reader2 = new
DataReader(protectedData.GetInputStreamAt(0));
        await reader1.LoadAsync((uint)inputData.Size);
        await reader2.LoadAsync((uint)protectedData.Size);
        IBuffer buffOriginalData =
reader1.ReadBuffer((uint)inputData.Size);
        IBuffer buffProtectedData =
reader2.ReadBuffer((uint)protectedData.Size);

        if (CryptographicBuffer.Compare(buffOriginalData,
buffProtectedData))
        {
            throw new Exception("ProtectStreamAsync returned unprotected
data");
        }

        // Return the encrypted data.
        return buffProtectedData;
    }

    public async Task<String> SampleDataUnprotectStream(
        IBuffer buffProtected,
        BinaryStringEncoding encoding)
    {
        // Create a DataProtectionProvider object.
        DataProtectionProvider Provider = new DataProtectionProvider();

```

```

        // Create a random access stream to contain the encrypted
message.
        InMemoryRandomAccessStream inputData = new
InMemoryRandomAccessStream();

        // Create a random access stream to contain the decrypted data.
        InMemoryRandomAccessStream unprotectedData = new
InMemoryRandomAccessStream();

        // Retrieve an IOutputStream object and fill it with the input
(encrypted) data.
        IOutputStream outputStream = inputData.GetOutputStreamAt(0);
        DataWriter writer = new DataWriter(outputStream);
        writer.WriteBuffer(buffProtected);
        await writer.StoreAsync();
        await outputStream.FlushAsync();

        // Retrieve an IInputStream object from which you can read the
input (encrypted) data.
        IInputStream source = inputData.GetInputStreamAt(0);

        // Retrieve an IOutputStream object and fill it with decrypted
data.
        IOutputStream dest = unprotectedData.GetOutputStreamAt(0);
        await Provider.UnprotectStreamAsync(source, dest);
        await dest.FlushAsync();

        // Write the decrypted data to an IBuffer object.
        DataReader reader2 = new
DataReader(unprotectedData.GetInputStreamAt(0));
        await reader2.LoadAsync((uint)unprotectedData.Size);
        IBuffer buffUnprotectedData =
reader2.ReadBuffer((uint)unprotectedData.Size);

        // Convert the IBuffer object to a string using the same
encoding that was
        // used previously to conver the plaintext string (before
encryption) to an
        // IBuffer object.
        String strUnprotected =
CryptographicBuffer.ConvertBinaryToString(encoding, buffUnprotectedData);

        // Return the decrypted data.
        return strUnprotected;
    }
}
}

```